HANDBOOK

Artificial Intelligence Assistant

Grade IX – Vocational Education

AI Concepts • Scratch • Python • Data • Machine Learning

A textbook-style learning resource with worked examples, diagrams and review questions

Based on the supplied Grade IX study material

Contents

1 Introduction to Artificial Intelligence **1**
1.1 What is artificial intelligence? . . . 1
1.2 Core abilities associated with AI . . . 1
1.3 AI in everyday life . . . 1
1.4 Why learn AI at school? . . . 2
1.5 Types of Artificial Intelligence . . . 2
1.5.1 Types based on capability . . . 2
1.5.2 Types based on how a system works . . . 2
1.6 Responsible use of AI . . . 3
1.7 Chapter questions . . . 3

2 Programming Basics and Algorithms **4**
2.1 What is programming? . . . 4
2.2 Algorithms and logical steps . . . 4
2.3 Sequence, selection and repetition . . . 4
2.4 Variables, input and output . . . 4
2.5 Testing and debugging . . . 5
2.6 Chapter questions . . . 5

3 Coding with Scratch **6**
3.1 Introducing block-based programming . . . 6
3.2 The Scratch workspace . . . 6
3.3 Events and motion . . . 6
3.4 Control and sensing . . . 6
3.5 Planning a simple project . . . 6
3.6 Common errors . . . 7
3.7 Chapter questions . . . 7

4 Introduction to Python **8**
4.1 What is Python? . . . 8
4.2 A first program . . . 8
4.3 Comments and indentation . . . 8
4.4 Reading input and displaying output . . . 8
4.5 Errors in Python . . . 8
4.6 Chapter questions . . . 8

5 Data Types and Operators **10**
5.1 Why data types matter . . . 10
5.2 Arithmetic operators . . . 10
5.3 Comparison and logical operators . . . 10
5.4 Small worked example . . . 10
5.5 Chapter questions . . . 10

6 Basics of Data Literacy **11**
6.1 Understanding data . . . 11
6.2 Sources and types of data . . . 11
6.3 A basic data workflow . . . 11
6.4 Reading a dataset . . . 11
6.5 Privacy and fairness . . . 11
6.6 Chapter questions . . . 11

7 Data Pre-processing **12**
7.1 Why data need preparation . . . 12
7.2 Common data-quality issues . . . 12
7.3 A careful cleaning process . . . 12
7.4 Preparing data for machine learning . . . 12
7.5 Chapter questions . . . 12

8 Mathematics for AI: Data Analysis **13**
8.1 Why mathematics is useful . . . 13
8.2 Counts, totals and averages . . . 13
8.3 Percentages and ratios . . . 13
8.4 Correlation and caution . . . 13
8.5 Chapter questions . . . 13

9 Data Visualisation **14**
9.1 Choosing a chart . . . 14
9.2 Make a chart understandable . . . 14
9.3 Interpreting visual information . . . 14
9.4 Chapter questions . . . 14

10 Introduction to Machine Learning **15**
10.1 What is machine learning? . . . 15
10.2 A typical machine-learning workflow . . . 15
10.3 Features, labels and models . . . 15
10.4 Overfitting and evaluation . . . 15
10.5 Limits of machine learning . . . 15
10.6 Chapter questions . . . 15

11 Types of Machine Learning **16**
11.1 Supervised learning . . . 16
11.2 Unsupervised learning . . . 16
11.3 Reinforcement learning . . . 16
11.4 Deep learning . . . 16
11.5 Selecting an approach . . . 16
11.6 Chapter questions . . . 17

Chapter 1

Introduction to Artificial Intelligence

1.1 What is artificial intelligence?

Artificial Intelligence (AI) is a field of computing concerned with building systems that perform tasks commonly associated with human intelligence. These tasks may include recognising patterns, understanding language, interpreting images, making predictions and selecting actions. An AI system does not have to think exactly like a person; it is designed to perform a particular task using data, rules or learned patterns.

The word **artificial** means made by people, while **intelligence** includes abilities such as learning, reasoning, understanding information and solving problems. A spam filter, a voice assistant, a translation service and a system that recognises objects in photographs are everyday examples of AI-supported applications.

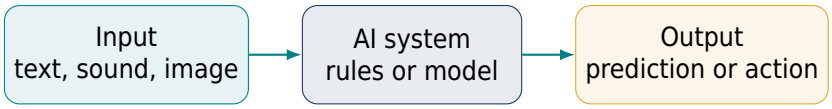


Figure 1.1. A simplified view of how an AI application receives input and produces an output.

1.2 Core abilities associated with AI

- **Learning:** using examples or experience to improve performance on a task.
- **Reasoning:** applying rules or relationships to reach a conclusion.
- **Problem solving:** considering possible steps and selecting a suitable solution.
- **Perception:** interpreting images, sound or other signals from the environment.
- **Language processing:** working with written or spoken human language.

Not every AI application uses all these abilities. A simple rule-based system may follow conditions written by a developer, while a machine-learning system may learn useful patterns from examples.

1.3 AI in everyday life

AI-supported features can appear in email spam detection, maps and route suggestions, speech recognition, translation, accessibility tools, shopping recommendations, fraud alerts and image search. In each case, the system uses input data and a designed method to produce a result. The result may be helpful, but it can still be incomplete or incorrect.

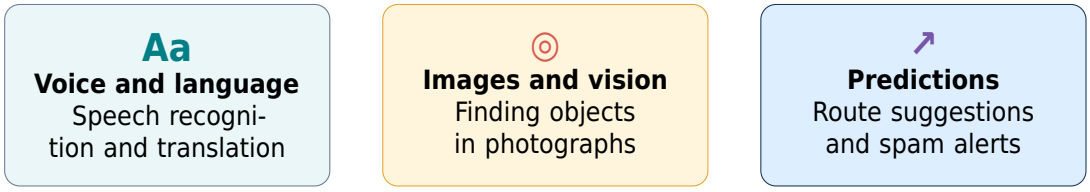


Figure 1.4. Examples of everyday tasks where AI-supported features may be used.

1.4 Why learn AI at school?

Learning AI helps students understand digital services they already use and develop the vocabulary needed to discuss them. It also builds habits of logical thinking, careful observation, data interpretation and responsible technology use. Students can begin with visual programming and gradually move towards text-based programming and data projects.

1.5 Types of Artificial Intelligence

AI can be grouped in more than one way. One common classroom explanation groups it by the range of tasks it could perform. Another describes how a system behaves or uses information. These are useful learning frameworks, not proof that every category already exists as a real product.

1.5.1 Types based on capability

- **Narrow AI (ANI):** designed for a specific task or a limited set of related tasks. Examples include speech recognition, spam filtering and route suggestions. Most AI applications used today fall into this category.
- **Artificial General Intelligence (AGI):** a proposed system that could learn and perform a broad range of intellectual tasks at a flexible, human-like level. It has not been established as a generally available system.
- **Artificial Superintelligence (ASI):** a hypothetical concept describing AI that would exceed human abilities across many important areas. It is an idea discussed in research and public debate, not an established current technology.

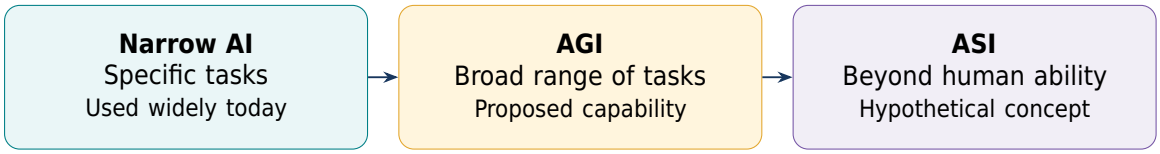


Figure 1.2. Capability categories, from task-specific AI to broader hypothetical capabilities. The arrows show a conceptual progression, not a guaranteed development path.

1.5.2 Types based on how a system works

- **Reactive machines:** respond to the current input using a defined method, without storing experiences to improve future decisions. They are a simple conceptual category.
- **Limited-memory AI:** uses recent data or information from past observations to support a task. Many modern AI applications use some form of stored context or historical data, although the details differ.
- **Theory-of-mind AI:** a proposed type that would interpret other people's beliefs, intentions or emotions in a deeper, context-aware way. Current systems may recognise cues but can misunderstand what people actually think or feel.
- **Self-aware AI:** a hypothetical system with genuine self-awareness. There is no established evidence that today's AI systems are self-aware.

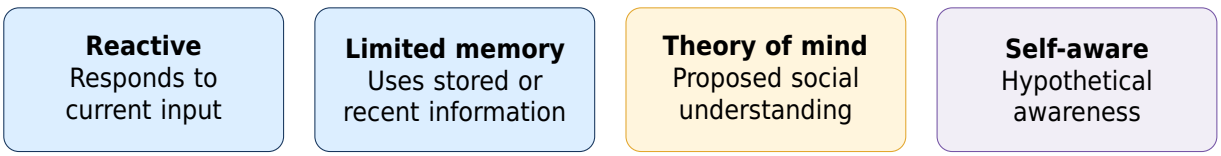


Figure 1.3. A simplified comparison of functional categories often used to introduce AI. The last two categories remain theoretical.

1.6 Responsible use of AI

AI outputs should be checked, particularly when a decision affects a person's safety, education, finances or opportunities. Avoid entering private information into tools unless their data practices are understood and authorised. Consider fairness, privacy, transparency, accessibility and the possibility that a model may make mistakes. Human judgement remains important.

1.7 Chapter questions

1. Define artificial intelligence in your own words.
2. Name four abilities associated with AI and give one example for each.
3. Explain two uses of AI in everyday life.
4. Name the three capability-based types of AI and give one-line descriptions.
5. Which functional categories of AI are considered hypothetical, and why?
6. What is the difference between narrow AI and the idea of AGI?
7. Why should AI-generated results be checked before they are used?

Chapter 2

Programming Basics and Algorithms

2.1 What is programming?

Programming is the process of writing instructions that a computer can execute. A program translates a solution into steps that are precise enough for a computer to follow. Programming is used to create websites, games, mobile applications, data tools and AI systems.

2.2 Algorithms and logical steps

An **algorithm** is a finite, ordered set of steps for solving a problem. Before writing code, it is useful to state the problem, identify the required input, decide the processing steps and describe the expected output. A clear algorithm is easier to test and improve than a collection of unplanned instructions.

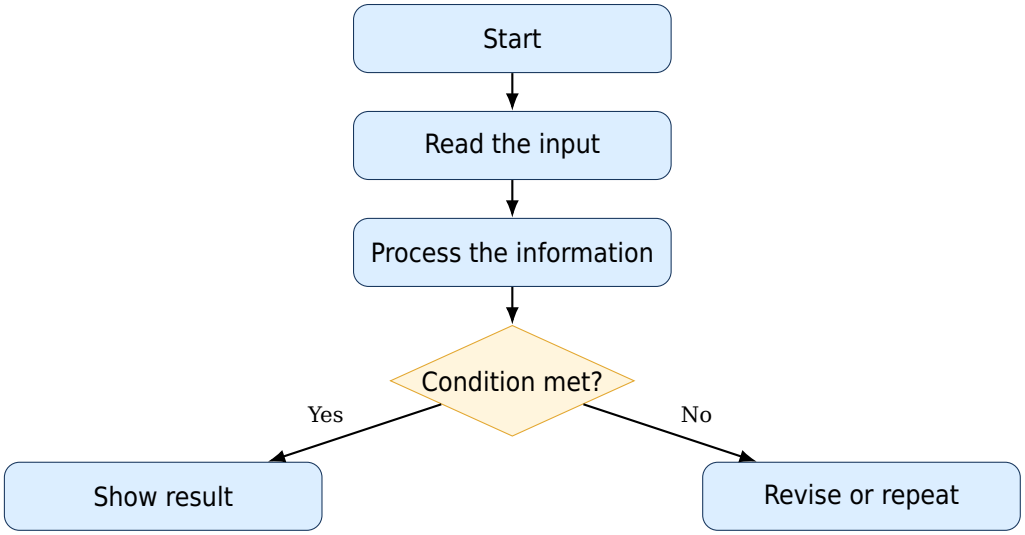


Figure 2.1. A flowchart shows the order of steps and decisions in an algorithm.

2.3 Sequence, selection and repetition

Sequence means instructions run in order. **Selection** chooses between alternatives using a condition, such as whether a value is greater than zero. **Iteration** repeats a set of instructions until a condition is met or a fixed number of repetitions has completed. These three structures are common in many programming languages.

2.4 Variables, input and output

A variable is a named place for a value used by a program. Input is information supplied to the program; output is the result presented to the user or another system. For example, a program may read two numbers, add them and display the sum. Meaningful variable names and consistent indentation make code easier to read.

2.5 Testing and debugging

Testing checks whether a program behaves as expected. Try ordinary values, boundary values and unexpected inputs. Debugging is the process of locating and correcting the cause of a problem. A useful approach is to reproduce the error, inspect the relevant steps, change one thing at a time and test again.

2.6 Chapter questions

1. What is an algorithm? Why should it be planned before coding?
2. Explain sequence, selection and iteration with examples.
3. What is the purpose of a variable?
4. Distinguish testing from debugging.
5. Draw a flowchart for deciding whether a number is positive or not positive.

Chapter 3

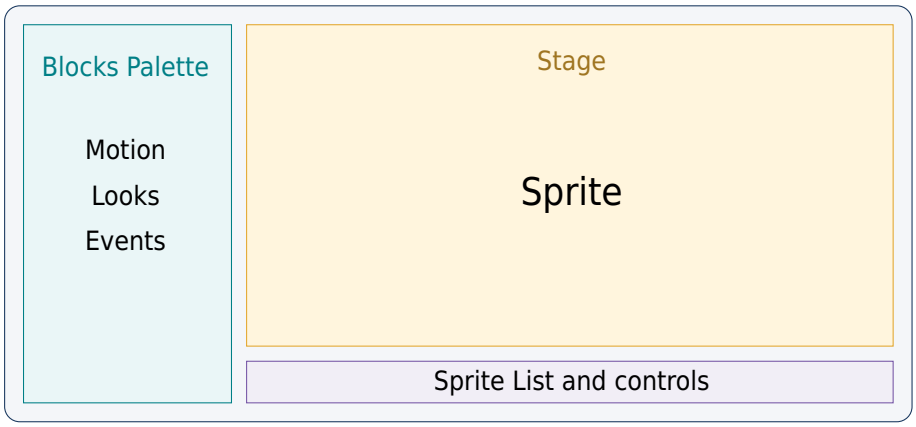
Coding with Scratch

3.1 Introducing block-based programming

Scratch is a visual programming environment in which programs are assembled from connected blocks. It helps learners explore computational thinking without needing to memorise punctuation and syntax at the beginning. A project can include sprites, backdrops, sounds, events, motion, looks, sensing and control blocks.

3.2 The Scratch workspace

The Stage displays the project while sprites are the characters or objects that perform actions. The Blocks Palette contains programming blocks, the Code Area is where blocks are connected, and the Sprite List helps select and manage project objects. A backdrop changes the Stage background.



Illustrative layout of a block-based programming workspace

3.3 Events and motion

An event block starts a script when something happens, such as clicking the green flag or pressing a key. Motion blocks move a sprite, turn it or change its position. Looks blocks change appearance, costumes or speech bubbles. Combining event and motion blocks can create a simple interactive animation.

3.4 Control and sensing

Control blocks repeat actions, wait for a period or run instructions only when a condition is true. Sensing blocks detect events or information, such as whether a sprite touches another object. Operators perform comparisons and calculations. Variables can store a score, a timer or the number of attempts.

3.5 Planning a simple project

For a simple catching game, define the goal first: the player moves a sprite to catch a falling object. Decide which keys control movement, how a catch is detected, how the score changes and when the game ends. Build one feature at a time and test it before adding sound or extra levels. Keep the rules clear so another learner can understand the game.



Figure 3.1. A simplified Scratch script: an event starts the action, then motion instructions run. Real Scratch blocks have their own shapes and colours.

3.6 Common errors

Disconnected blocks do not run as part of the intended script. A script may begin with the wrong event, repeat forever or check a condition before the sprite reaches the target. Test with slow movement first, then adjust speed and timing. Give sprites and variables meaningful names.

3.7 Chapter questions

1. Name four main areas of the Scratch workspace.
2. What is the role of an event block?
3. How are control blocks different from motion blocks?
4. Describe the steps needed to plan a simple catching game.
5. Give two ways to find errors in a Scratch project.

Chapter 4

Introduction to Python

4.1 What is Python?

Python is a high-level programming language known for readable syntax. It is used for automation, web development, data analysis, scientific work and many AI projects. Python programs are written as text and executed by a Python interpreter. The exact interface depends on the installed environment.

4.2 A first program

The statement below displays a message:

```
print("Hello, AI!")
```

The function `print()` sends information to the output. Text placed inside quotation marks is a string. Parentheses group the value passed to the function.

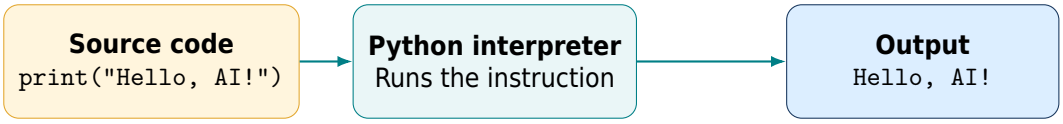


Figure 4.1. A short Python statement is interpreted and produces visible output.

4.3 Comments and indentation

A comment begins with the # symbol and is ignored when the program runs. Comments explain why a step is needed. Indentation marks the body of many Python structures, such as conditional statements and loops. Inconsistent indentation can cause an error, so use a consistent style.

4.4 Reading input and displaying output

```
name = input("Enter your name: ")
print("Welcome", name)
```

The `input()` function reads text from the user. A value read with `input()` is a string by default. If a program needs a number, it must convert the text carefully, for example with `int()` for a whole number.

4.5 Errors in Python

A syntax error means the code does not follow the language rules. A runtime error occurs while the program is executing. A logic error means the program runs but produces an incorrect result. Read the error message, check the line it identifies and test a small example to isolate the problem.

4.6 Chapter questions

1. What is Python used for?

2. Write a Python statement that prints your school name.
3. What is a comment, and how is it written in Python?
4. Why is indentation important?
5. Distinguish syntax, runtime and logic errors.

Chapter 5

Data Types and Operators

5.1 Why data types matter

A data type describes the kind of value a variable holds. Choosing a suitable type helps a program perform the intended operation and prevents avoidable errors. Common introductory Python types include integers, floating-point numbers, strings and Boolean values.

Type	Meaning	Example
`int`	Whole number	`students = 32`
`float`	Number with a decimal part	`temperature = 28.5`
`str`	Text	`city = "Bhopal"`
`bool`	True or false value	`is_ready = True`

5.2 Arithmetic operators

Python supports addition (+), subtraction (−), multiplication (*), division (/), floor division (//), remainder (%) and exponentiation (**). Use parentheses to make the intended order of operations clear. Division by zero is invalid and should be handled when user input is involved.

5.3 Comparison and logical operators

Comparison operators such as ==, !=, >, <, >= and <= produce Boolean results. Logical operators `and`, `or` and `not` combine or reverse conditions. They are often used in decisions and loops.

5.4 Small worked example

```
marks = 76
if marks >= 35:
print("Pass")
else:
print("Try again")
```

The condition is evaluated as true or false. The program prints one message depending on the result. In a complete application, the value should be read and validated rather than fixed in the source code.

5.5 Chapter questions

1. List four common Python data types.
2. Explain the difference between = and ==.
3. What result does the remainder operator produce?
4. Give an example of a logical condition using `and`.
5. Write a short program that checks whether a number is greater than zero.

Chapter 6

Basics of Data Literacy

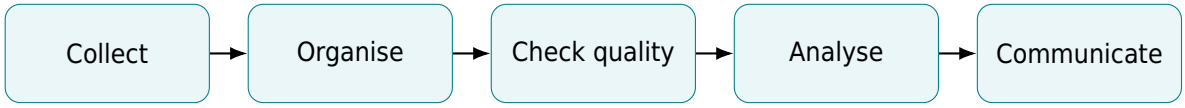
6.1 Understanding data

Data are recorded observations, values or descriptions. They may be numerical, textual, categorical, visual or audio-based. Information is produced when data are organised and interpreted in a context. Data literacy is the ability to read, question, analyse and communicate information from data responsibly.

6.2 Sources and types of data

Data may come from surveys, forms, sensors, observations, transactions or existing datasets. Primary data are collected directly for a purpose; secondary data were collected earlier by another person or organisation. Quantitative data describe counts or measurements. Qualitative data describe qualities or categories. Before using a dataset, identify its source, purpose, units, time period and any limitations.

6.3 A basic data workflow



6.4 Reading a dataset

A dataset arranged in rows and columns often uses each row for one record and each column for one variable. A column heading should clearly describe the values below it. Check whether units are consistent, whether values are missing and whether the same record appears more than once. A surprising value is a reason to investigate, not automatically a reason to delete it.

6.5 Privacy and fairness

Collect only data needed for a legitimate purpose. Do not publish personal details without appropriate permission. A dataset may under-represent a group or reflect past unfair decisions. Ask who is included, who is missing and how conclusions may affect different people. Good data practice includes secure storage, careful access and honest communication of uncertainty.

6.6 Chapter questions

1. Define data literacy.
2. Distinguish primary and secondary data.
3. Explain quantitative and qualitative data with examples.
4. Name four checks that can improve data quality.
5. Why should a dataset’s source and limitations be documented?

Chapter 7

Data Pre-processing

7.1 Why data need preparation

Collected data are often inconsistent or incomplete. Data pre-processing is the set of steps used to inspect and prepare data before analysis or machine learning. Poor-quality input can lead to misleading summaries and unreliable model predictions. Preparation should be appropriate to the question being studied.

7.2 Common data-quality issues

Missing values occur when a field has not been recorded. Duplicates represent repeated records. Inconsistent formats may use different spellings, date patterns or units. Invalid values fall outside the rules for a field. Outliers are unusually high or low values; they may be genuine observations or errors and require investigation.

7.3 A careful cleaning process

1. Keep an unchanged copy of the original dataset.
2. Inspect column names, data types, units and sample records.
3. Identify missing values, duplicates and inconsistent formats.
4. Decide how each issue should be handled and record the reason.
5. Validate the cleaned data and compare it with the original.

Do not automatically replace every missing value with zero: zero may have a real meaning. Depending on the task, a missing value may be left unknown, filled using a justified method or excluded with a clear explanation.

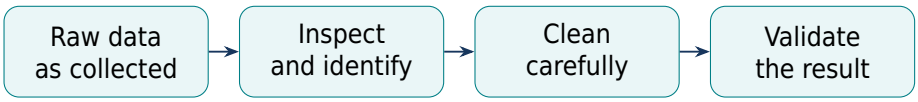


Figure 7.1. A simple data-cleaning workflow. Keep the original data and record important changes.

7.4 Preparing data for machine learning

For supervised learning, input features and the target label must be identified. Categorical data may need encoding and numerical features may need scaling for some algorithms. Split data into training and test sets before using test results to make decisions. Any transformation that learns from data should be fitted using the training data only, to reduce information leakage.

7.5 Chapter questions

1. What is data pre-processing?
2. Give three examples of data-quality problems.
3. Why should the original dataset be preserved?
4. Why is replacing every missing value with zero unsafe?
5. Explain the purpose of a training and test split.

Chapter 8

Mathematics for AI: Data Analysis

8.1 Why mathematics is useful

Mathematics helps describe data, compare values and reason about patterns. Basic arithmetic, fractions, percentages, averages, ratios and simple graphs are useful when interpreting data. Mathematical summaries do not replace context: the same number can mean different things depending on how data were collected.

8.2 Counts, totals and averages

The total is the sum of values. The mean is the total divided by the number of values. The median is the middle value after sorting, and the mode is the most frequent value. The mean can be affected by extreme values, while the median is often more resistant to them.

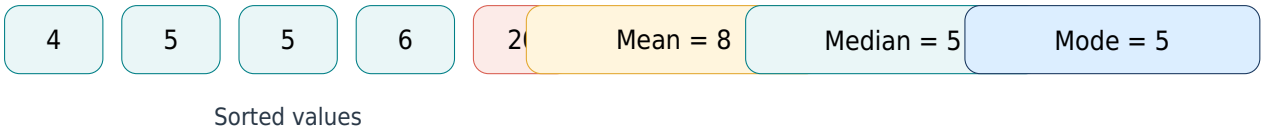


Figure 8.2. A compact visual summary of one dataset. The unusually large value 20 pulls the mean above the median and mode.

8.3 Percentages and ratios

A percentage expresses a part out of one hundred. The percentage of a group meeting a condition is calculated as the number meeting the condition divided by the total number, multiplied by 100. A ratio compares quantities. Always state the denominator and the population being compared so that a percentage is not misinterpreted.

8.4 Correlation and caution

Two variables may change together, but this does not prove that one causes the other. A third factor may influence both, or the pattern may be coincidental. When interpreting data, distinguish observation from explanation and avoid claims that go beyond the evidence.

8.5 Chapter questions

1. Calculate the mean of 6, 8, 10 and 12.
2. Find the median of 3, 7, 9, 12 and 18.
3. How is a percentage calculated?
4. Why can the median be more useful than the mean for data with extreme values?
5. Explain why correlation does not by itself prove causation.

Chapter 9

Data Visualisation

9.1 Choosing a chart

Data visualisation presents information in a visual form so that patterns and comparisons can be noticed. A bar chart compares categories; a line chart shows change over an ordered sequence such as time; a pie chart may show parts of a whole when there are only a few categories; and a scatter plot shows the relationship between two numerical variables.

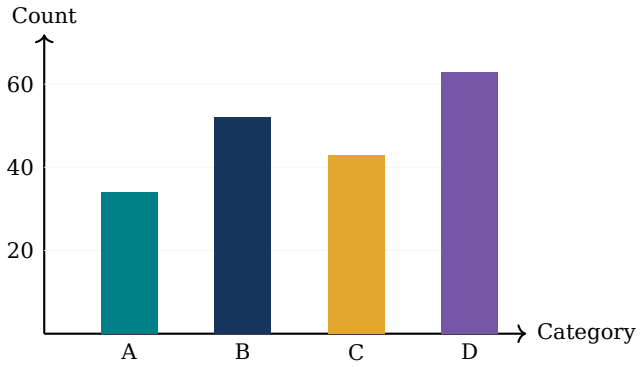


Figure 8.1. An illustrative bar chart. Values are examples, not measurements from the source material.

9.2 Make a chart understandable

Every chart should have a descriptive title, labelled axes where needed, clear units and a scale that does not mislead. Use a legend only when multiple series need identification. Avoid decorative effects that make comparisons difficult. If values are estimates or incomplete, explain that limitation.

9.3 Interpreting visual information

Read the title and labels first. Check the scale before comparing bar heights or slopes. Look for the largest and smallest values, changes over time and unusual points. Do not infer a cause from a chart alone. A chart should support a conclusion that can also be explained in words.

9.4 Chapter questions

1. Which chart is suitable for comparing categories?
2. When is a line chart useful?
3. Name four features of a clear chart.
4. How can a graph mislead a reader?
5. Why should the source and units of plotted data be stated?

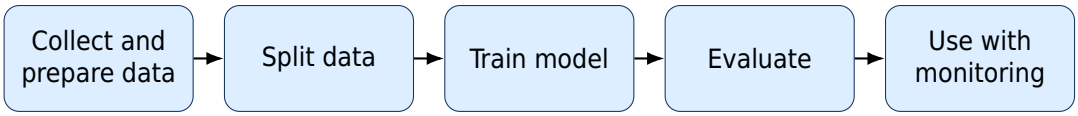
Chapter 10

Introduction to Machine Learning

10.1 What is machine learning?

Machine Learning (ML) is a part of AI in which a system uses data to learn patterns that help it make predictions or decisions. Instead of writing a separate rule for every possible case, a developer chooses a method, supplies suitable examples and evaluates the resulting model. ML performance depends on the data, the task and the way the model is tested.

10.2 A typical machine-learning workflow



10.3 Features, labels and models

A **feature** is an input used by a model, such as the size or colour of an object. A **label** is the answer associated with an example in a labelled dataset. A **model** is the learned pattern or relationship used to produce an output. Training adjusts the model based on examples; evaluation checks how well it works on data not used for training.

10.4 Overfitting and evaluation

A model may memorise details of training examples instead of learning patterns that generalise. This is called overfitting. Testing with separate data gives a more realistic indication of performance on new cases. Accuracy can be useful, but it may hide important errors when classes are imbalanced. Consider the consequences of false positives and false negatives for the task.

10.5 Limits of machine learning

ML systems can reflect gaps and bias in their training data. A model may perform well in one setting and poorly in another. Predictions are not guarantees. Systems used in real settings need monitoring, privacy safeguards, appropriate human oversight and a way to report problems.

10.6 Chapter questions

1. Define machine learning and explain its relationship to AI.
2. What are features, labels and models?
3. Why should evaluation data be kept separate from training data?
4. What is overfitting?
5. Give two limitations or risks of ML systems.

Chapter 11

Types of Machine Learning

11.1 Supervised learning

Supervised learning uses examples that include inputs and known target labels. A classification model predicts a category, such as whether an email is likely to be spam. A regression model predicts a numerical value, such as an estimated measurement. The quality of the labels and the relevance of the examples affect the result.

11.2 Unsupervised learning

Unsupervised learning works with data that do not have target labels supplied for each example. A method may group similar records or identify structure in the data. Clustering can help explore patterns, but the groups need interpretation and are not automatically meaningful categories.

11.3 Reinforcement learning

In reinforcement learning, an agent takes actions in an environment and receives feedback in the form of rewards or penalties. The agent learns a strategy that aims to improve long-term reward. This approach is used in some control, robotics and game-playing tasks. A reward design that does not match the real goal can lead to undesirable behaviour.

Learning type	Training information	Example task
Supervised	Inputs with known labels or target values	Classify messages as spam or not spam
Unsupervised	Inputs without target labels	Explore groups of similar records
Reinforcement	Actions, environment feedback and rewards	Learn a strategy in a simulated game

11.4 Deep learning

Deep learning uses neural networks with multiple processing layers to learn representations from data. It can be effective for images, audio, language and other complex inputs, but it often needs substantial data and computing resources. Deep learning is one family of methods within machine learning, not a synonym for all AI.

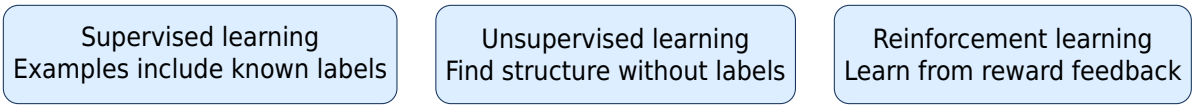


Figure 10.1. Three common learning approaches differ in the kind of feedback or information available.

11.5 Selecting an approach

Start with the problem and the available data. If examples have correct labels, supervised learning may be suitable. If the aim is to explore structure without labels,

an unsupervised method may help. If actions influence an environment and feedback is available, reinforcement learning may be relevant. The choice should be tested rather than assumed.

11.6 Chapter questions

1. Explain supervised learning with an example.
2. How does unsupervised learning differ from supervised learning?
3. What role does a reward play in reinforcement learning?
4. What is deep learning?
5. Match a suitable learning type to a spam filter, customer grouping and a game-playing agent.